

FlashANNs: Hiding NAND from Graph Search on Flash-Backed CXL Memory

Anonymous Author(s)

Abstract

Billion-scale graph ANNS needs a flash-priced corpus and a hot path a query host can score from. Block-SSD systems such as DiskANN hide NAND behind asynchronous I/O and PQ navigation, but remain a per-host block serving unit. Full-in-memory CXL-DRAM systems such as CXL-ANNS and Cosmos remove the flash miss entirely and price the cold tail at DRAM. Flash-backed CXL memory (CXL-SSD) is the third point: NAND capacity behind a small memory-semantic window. A unified VA does not make search usable. The runtime question is what must be hidden so that graph walk does not wait on NAND.

We present FlashANNs, a graph-ANNS runtime that splits that contract. *Score hide*—every full-precision distance comes from bytes already in the window—is a failure if the scoring instruction touches NAND. *Query hide*—the timed path also does not wait for a fill—is the oracle, not a result we claim. Graph and PQ live in host DRAM; only soon-to-expand frontier pages enter the window; fills are asynchronous and issued across queries so one pointer-chasing walk does not collapse device depth to one. On true-cold Text2Image-10M (recall@10 ≥ 0.92 , $L=400$), score hide holds (from.win=100%) at 50.25 QPS versus 1.64 QPS demand, but remains $0.59\times$ an in-window oracle (85.8 QPS) because hop $i+1$ cannot issue until hop i scores. A same-machine PipeANN block path is a named residual (oneshot full-precision versus PQ navigation). We evaluate an FPGA-attached NVMe plus a software memory-semantic proxy (vmem_sw), do not claim a product CXL.mem SSD or 160 GB/s, and treat multi-host one-copy as an unevaluated Shared-FAM deployment model.

CCS Concepts: • Computer systems organization → Secondary storage organization; • Information systems → Nearest-neighbor search; • Software and its engineering → Memory management.

Keywords: CXL, CXL-SSD, approximate nearest neighbor search, graph ANNS, memory-semantic flash, residency, prefetch, continuous batching

1 Introduction

Graph-based approximate nearest neighbor search (ANNS) is a production serving primitive: a query walks a proximity graph and scores neighbors until a beam budget is spent. At billions of vectors the graph plus full-precision

embeddings no longer fit in host DRAM, so the corpus must live behind another tier. CXL makes that tier addressable with `load/store`. When the whole corpus fits in CXL-DRAM, prior systems already offer a usable service—CXL-ANNS and Cosmos [4, 14, 15] never stall on flash. This paper is about the next device. Flash-backed CXL memory (CXL-SSD) exposes NAND capacity and only a small DRAM-fast window on a CXL.mem address space [3, 7, 10, 13]. A unified VA does not make graph search usable: a window miss costs tens of microseconds. To our knowledge, FlashANNs is the first *graph-ANNS serving runtime* that operates directly over an SSD-backed CXL address space and makes *scoring* usable—the host computes full-precision distances from bytes that already sit in that window. We do not claim the first CXL-DRAM ANNS, the first CXL-SSD attached to a vector database, or a product CXL.mem SSD.

Two published lines each solve one side of the capacity-latency tension and tax the other (Figure 1). DiskANN [18, 19] and pipelined descendants such as PipeANN [9] are excellent *block-SSD* ANNS: PQ navigation, sector layout, and asynchronous I/O hide NAND behind a per-host cache. Their serving unit remains CPU + DRAM working set + a local SSD index. Adding R query workers typically replicates provisioned flash or shards the graph. They hide *block I/O*, not a faulting CXL address space. CXL-ANNS and Cosmos put the graph and embeddings in a CXL-DRAM pool. Search never sees flash, and a pooled HDM can be one copy—but the cold tail $D - H$ is priced at DRAM [1, 2]. We build a third service because neither line is the right contract for CXL-SSD: the corpus should stay flash-priced, the hot path should stay memory-semantic, and the runtime must hide NAND from the scoring instruction. Multi-host one-copy under Shared FAM is a deployment path we do not evaluate (§2).

Making that service real is hard because the device is not slightly slow DRAM and not a DiskANN SSD. Three hardware facts break conventional designs (§3). (C1) *The miss cliff*. A window hit is a DRAM-class copy; a miss is a NAND fill ($\sim 40\text{--}80\mu\text{s}$, about $140\times$). Graph walk multiplies the tax by hops $\times$ degree, so treating CXL-SSD as mmap’d memory makes search miss-bound. (C2) *Prefetch must be wise*. Neighbor IDs are not sequential. OS-style page admission and synchronous 2-hop graph prefetch bring vectors the walk will not score, pollute the small window, and can drop QPS

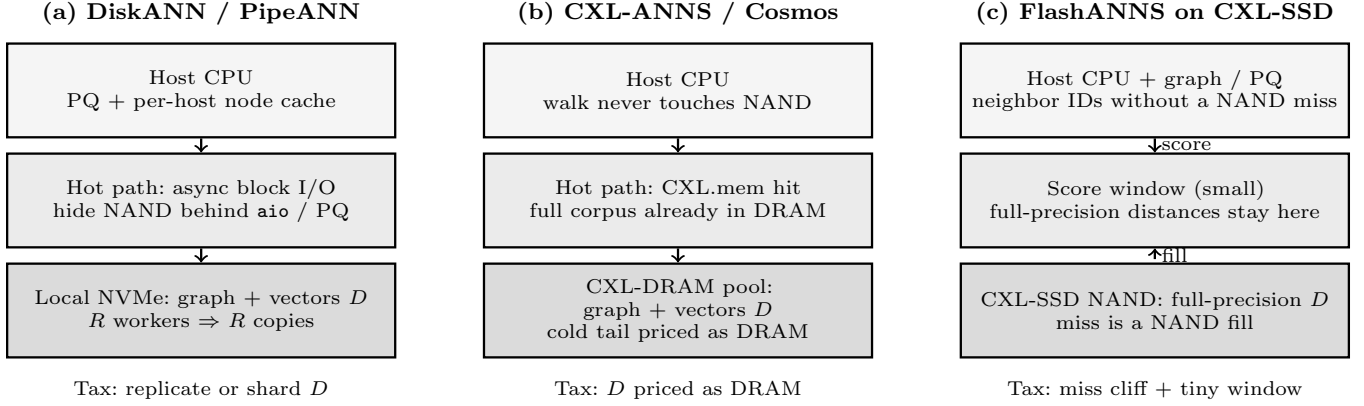


Figure 1. Two published deployments and the third point this paper serves. CXL-SSD is the *opportunity* (flash-priced D , a small memory-semantic window). FlashANNS is the *runtime*: a wise prefetcher plus continuous batching so scoring does not wait on NAND. The window in (c) is the score working set, not a measured CXL-DRAM DAX. Shared FAM is not a measured link.

by 3–5 $\times$. Wrong pages cost more than a missed page. (C3) *One query cannot fill the device.* Best-first search cannot issue hop $i+1$ ’s vector pages until hop i scores. A blocking load/store miss collapses queue depth to ≈ 1 ; a single walk leaves the NAND pipeline idle even when bandwidth remains. Continuous issue across queries is what holds depth; it does not, by itself, make one walk as fast as an in-window oracle.

FlashANNS is a runtime for those three constraints, not a new graph index. A placement premise comes first: the adjacency list and any PQ codebook live in host DRAM so the prefetcher can read neighbor IDs without a NAND miss; full-precision vectors stay on CXL-SSD. The first mechanism is a *wise prefetcher*. It issues asynchronous fills only for pages the current frontier is about to expand, packs co-scored neighbors into the same pages (expand-bundle / page-bin), and admits them under a byte budget. It never synchronous-walks a 2-hop neighborhood and never scores unused siblings to inflate utilization. The second mechanism is *continuous batching*. Fills are issued at fetch granularity across in-flight queries so that when one walk is stuck on pointer-chasing, another query keeps the device busy. Prefetch and batching may touch CXL-SSD; the scoring thread may not. We call that split *score hide*. The stronger contract—*query hide*, no wait for a fill on the timed path—is the oracle, not a result we claim. On true-cold Text2Image-10M (recall@10 ≥ 0.92 , $L=400$), score hide holds (from_win=100%) at 50.25 QPS versus 1.64 QPS demand, but remains 0.59 $\times$ an in-window oracle (85.8 QPS) because hop $i+1$ cannot issue until hop i scores.

This paper makes four contributions:

1. **A third serving point, scoped.** We place graph ANNS on flash-backed CXL memory and separate the *opportunity* (flash-priced D , a small window) from the *system* (hide NAND from scoring). We do not claim the first CXL-DRAM ANNS or an evaluated Shared-FAM deployment (§2).
2. **Why naive CXL-SSD ANNS fails.** We measure the miss cliff, unwise page/prefetch admission, and single-query queue collapse, and we turn those negatives into design constraints (§3).
3. **A wise prefetcher plus continuous batching.** Placement keeps neighbor IDs off the window; the prefetcher fills only soon-to-expand pages; batching holds device depth across walks. Score hide reaches 100% window hits on the scoring path; query hide remains 0.59 $\times$ the in-window oracle (§4, §5).
4. **An iso-recall evaluation of hide, not just QPS.** We report scores-from-window, critical-path wait, and QPS from demand (1.64) through FlashANNS (50.25) to the oracle (85.8). Same-machine PipeANN is a named residual (oneshot full-precision versus PQ navigation), not a hidden bar (§6).

2 Background

2.1 Graph ANNS walk

A graph ANNS index (Vamana [19], HNSW [16]) stores, for each vector, a neighbor list of degree R . Search keeps a candidate set of width L (beam / efSearch). A *hop* expands the closest unread candidate u : the walk reads $N(u)$, scores those neighbors, and inserts the best unseen IDs back into the candidate set. The query returns the top- k after a hop or candidate budget is spent.

One expand touches two kinds of bytes. Neighbor *IDs* are small and can live in host DRAM. Neighbor *vectors* are large: on Text2Image-10M each is 800 B, so scoring $N(u)$ moves about $R \times 800$ B if every neighbor is new. A typical hop is:

1. pick the closest unread candidate u from the beam;
2. read $N(u)$ (IDs only);
3. score the unseen neighbors' full-precision vectors;
4. insert the best new IDs into the candidate set.

Two facts follow for a memory hierarchy. First, the walk is *pointer-chasing*: step 3 of hop i decides which vector pages hop $i+1$ will need. Second, IDs need not live with vectors—the adjacency list can sit in host DRAM while embeddings sit on a slower tier. Quality rises with how many useful neighbors are scored; stall rises with how many of those scores miss the fast tier. Widening L scores more nodes and cools the working set; that coupling is a measurement in §3, not a claim here.

2.2 The block-SSD serving stack

DiskANN [18, 19] is the reference block-SSD design. Full-precision vectors and adjacency share 4 KB-aligned NVMe sectors. A compact PQ codebook in DRAM ranks neighbors without a NAND read; an *aio* beam issues several sector reads at once; a static node cache pins entry vertices. PQ navigation exists to *avoid* full-precision reads, not to make NAND faster. Asynchronous I/O *hides* remaining reads by overlapping them with other work. Because graph and vectors share a sector, one 4 KB read is bound to whatever else the layout packed into that page.

PipeANN [9] keeps that layout, overlaps compute with I/O inside one query, and can interleave queries to hold device queue depth. That interleaving is continuous batching *on a block device*: the first-class objects are the I/O queue and the sector. The same idea will reappear on CXL-SSD as fetch-level issue (§4), against a faulting address space rather than *aio*.

2.3 CXL-DRAM ANNS

CXL.mem maps device HDM into the host physical address space [13]. Published Type-3 CXL-DRAM latencies sit in the 140–410 ns range. CXL-ANNS [14, 15] places the graph and embeddings in a CXL-DRAM pool and hides that far-memory delay with a relationship-aware cache and DSA-assisted prefetch. Cosmos [4] further offloads traversal and distance to device-side compute. Second-tier and residual designs [1, 11, 17] likewise assume a DRAM- or NVM-priced far tier.

The interface is *load/store*; the capacity assumption is that D fits in CXL-DRAM. A service on that line is already usable in the sense that search never waits on flash. Prefetch and cache policies tuned for hundreds of

Table 1. Latency tiers that a graph walk can see. CXL-DRAM is the published range; NAND fill is the order on the path we measure. The cliff itself is §3.

Access	Order of magnitude	Where
Host DRAM load	~80–120 ns	graph / PQ, if placed t
Score-window hit	DRAM-class copy	resident full-precision b
CXL-DRAM (literature)	~140–410 ns	CXL-ANNS / Cosmos p
NAND fill (this path)	~20–80 μ s	window miss $\rightarrow$ flash

nanoseconds do not transfer to a NAND fill of tens of microseconds. We keep the detailed contrast in §7.

2.4 Flash-backed CXL memory

A CXL-SSD pairs NAND capacity with a modest DRAM cache and presents a byte-addressable window [3, 7, 10]. Figure 1c is the deployment we serve. A *score-window hit* is a DRAM-class copy. A *miss* is a NAND fill into that window, then a retry. The programming model looks like memory; the miss tax looks like flash. Table 1 places the three published tiers next to the NAND fill this paper must hide. The measured cliff belongs in §3; the table only fixes the orders of magnitude.

The window is bounded, and the admission unit is a page. On Text2Image-10M a vector is 800 B, so a 4 KB page holds several records—or several unused siblings. A blocking *load/store* miss and an explicit prefetch ioctl are both fills; they differ in *who waits*. If the scoring thread issues the fill, NAND is on the critical path. If a prefetcher issues it before the hop scores, the fill can leave that path. That split is the runtime contract in §4, not a result.

The prototype we measure is a software memory-semantic node (*vmem.sw*) in front of NVMe, with the score window implemented as host DRAM (*mbind*). It preserves fill semantics (a miss moves NAND bytes into the window). It does not preserve a product CXL.mem device latency or a CXL-DRAM DAX region. Hardware and generic OS work on this tier does not manage graph-ANNS residency; those systems belong in §7. Implementation limits belong in §5. Direct-attach CXL-SSD already gives one host flash-priced memory semantics; that is the deployment we measure. Multi-host one-copy under CXL 3.x Shared FAM is a premise we do not evaluate.

3 Motivation

Section 2 defined the walk and the fill. This section measures what happens if that fill is treated as a slightly slow DRAM load or as a DiskANN sector read. Each measurement answers two questions: does the effect exist, and how much does the default cost? The three results are the constraints Intro named C1–C3 (Table 2).

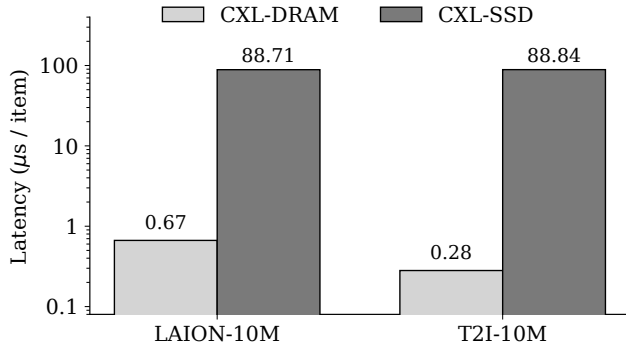


Figure 2. One-item host load: CXL-DRAM hit versus CXL-SSD cache miss. CXL-DRAM is Montage /dev/dax0.0 (clflush then load). CXL-SSD is a vmem_sw window miss. T2I-10M uses the staged 800 B vectors. LAION-10M is the first 10M items of the 25M dump ($d=512$); the miss bar matches that item size on the same SSD path.

3.1 The miss cliff

A host load of one vector from CXL-DRAM is a few hundred nanoseconds. The same load after a CXL-SSD window miss is a NAND fill (Figure 2). On Text2Image-10M (800 B) the p_{50} is $0.28 \mu s$ versus $89 \mu s$ ($\sim 320\times$). On a 10M-item LAION slice (2048 B) it is $0.67 \mu s$ versus $89 \mu s$ ($\sim 130\times$). The miss is a 4KB fill, so record size barely moves it; the CXL-DRAM hit scales with bytes.

Graph search multiplies the tax by hops $\times$ degree. Once a modest fraction of scores miss, wall-clock time is in the NAND path, not in distance arithmetic. DiskANN already knew the cliff for block I/O. The new fact is that a memory-semantic interface does not remove it: **load/store** only changes *how* a miss is issued (§3.3). Treating CXL-SSD as mmap’d DRAM therefore makes search miss-bound. The runtime contract that follows is not “make NAND faster”; it is that the scoring instruction must not wait on a fill.

3.2 Prefetch must be wise

Neighbor IDs are not sequential in the address space, so the next vector page is not the next byte. Admitting 16 KB pages (or cachelines grouped as pages) therefore brings siblings the walk will not score. On a high-dimensional corpus where a record already fills most of a page, per-record admission versus page admission splits QPS by $\sim 3.6\times$ and NAND reads by $\sim 3.7\times$ (Figure 3a). At small dimensions several records share a page and the gap narrows; page-cache thinking is still the wrong default here.

Synchronous logical prefetch of a 1-, 2-, or 8-hop neighborhood does not fix the cliff. Hit rate is almost unchanged, NAND reads multiply, and QPS drops by

$\sim 4\text{--}5\times$ at the widest setting (Figure 3b). Precision (*used/promoted*) falls as coverage grows; leftover pages evict useful ones from a small window. Wrong pages cost more than a missed page. The useful policy is a bounded ensure of the nodes the search is about to expand—not “prefetch harder.”

Widening the beam is the same mistake with a quality knob. Raising L grows the unique IDs touched per query, cools the window, and raises NAND reads faster than it raises recall (Figure 3c). Recall saturates once a hop budget is spent; further width is only stall. A controller that chops L on instantaneous miss rate does not, by itself, win at iso-recall: it has no recall constraint and no IO budget. Quality and stall must be read together. That is a measurement discipline, not a third design module.

3.3 One query cannot fill the device

Best-first search is pointer-chasing: hop i must score before the walk knows which vector pages hop $i+1$ needs. A single query therefore leaves device-queue bubbles—the beam ramps slowly, the next fill is unknown, and the walk drains at the end (Figure 4). On a block device, interleaving independent queries raises occupancy because the first-class object is already an I/O queue. On CXL-SSD the programming model is a faulting address space. A blocking **load/store** (or a blocking **memcpy** from the mapping) is a synchronous NAND fault: effective queue depth collapses to ≈ 1 per thread. Issuing speculative fills to keep the device busy, then scoring those pages on the critical path, pays the cliff again.

The idle device is a pipeline problem, not a bandwidth wall. Continuous issue across queries is what can hold depth; it does not, by itself, make one walk as fast as an in-window oracle, and it does not change which nodes the walk visits. The implementation of that issue belongs in §4.

3.4 From measurements to constraints

Table 2 is the contract for §4. We do not treat CXL-SSD as uniform memory (C1): scoring must not touch NAND. Prefetch must be wise (C2): only the soon-to-expand frontier, packed so a fill carries co-scored neighbors, under a byte budget. One walk cannot fill the device (C3): issue must be asynchronous and cross-query so pointer-chasing bubbles do not collapse queue depth.¹ Placement of neighbor IDs in host DRAM is a premise for C2, not a separate challenge: without it the prefetcher misses once just to learn whom to fetch.

¹A small shared pin of entry vertices helps a narrow beam more than deepening one walk. That is an admission hint, not a positive LFU-hot-set module, and not a fourth challenge.

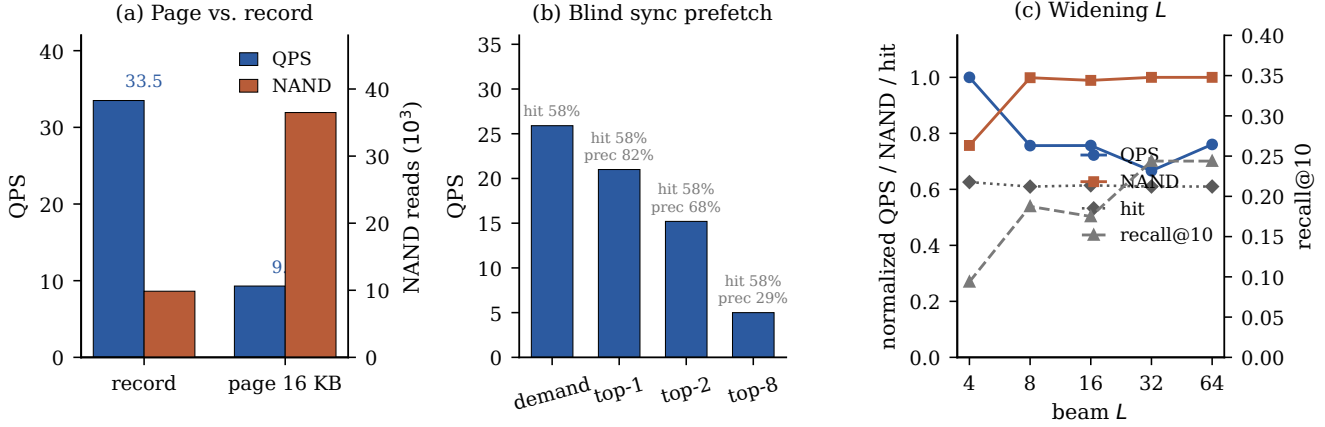


Figure 3. Unwise admission amplifies the cliff (LAION-200k, $d=1024$, hops=32). (a) Page vs. record. (b) Blind synchronous graph prefetch. (c) Widening L (coverage as another form of unwise). Absolute recall in (c) is low because of the hop cap; the 32→64 plateau is the claim.

Table 2. Measured pathologies become the three constraints Intro named. Deltas are from LAION-200k on the software CXL path; the qualitative claims are what Design must answer.

ID	Exists + hurts	Constraint	FlashANNS answers
C1	Miss cliff $\sim 130\times$	Scoring must not wait on NAND	Score-hide contract
C2	Page $\sim 3.6\times$; blind prefetch $\sim 4\text{--}5\times$	Prefetch only the expand frontier	Wise prefetcher
C3	Pointer-chasing; blocking fill $\Rightarrow$ QD ≈ 1	Hold depth across walks	Continuous batching

4 Design

FlashANNS is a userspace ANNS runtime plus a thin residency plane in front of host DRAM, a bounded window that plays the role of CXL-DRAM, and a flash-backed CXL address space that holds D (Figure 5). Search is still best-first graph walk. The service contract is a *split*: prefetch and continuous batching may touch CXL-SSD; the scoring thread may not. A hop that computes a full-precision distance from bytes that were not already in the window is a hide failure, not a successful promote.

One walk cannot fully hide hop $i+1$ until hop i scores (pointer-chasing). Intra-query hide is lookahead of the current frontier only. Inter-query continuous issue is what keeps device depth up while a single walk is stuck. We do not claim the measured prototype has met this contract (§6); the modules below are the mechanisms that must.

Table 3. Default placement. **Fill-in:** bytes on the corpus you report (e.g., $25M \times \text{dim}$).

Object	Default tier	Why
Graph (R neighbors)	host DRAM	every hop; keep off the win
PQ / codebook	host DRAM	tiny, extremely hot
Query / visited	host DRAM	per-worker state
Full-precision vec.	CXL-SSD	capacity
Entry / soft-pin	window (tiny)	F7; LFU hot set denoted
Frontier-top nbrs	window via prefetch	F3, F5; score only 121

4.1 D1: Semantic placement

Table 3 is the default map. The graph adjacency and the PQ codebook are touched on every hop and are small relative to the full-precision corpus; they live in host DRAM so they never compete with vectors for the CXL-SSD window (F2, F7). Query state (candidates, visited set, thread-local arenas) is likewise host-local. Full-precision vectors default to the CXL-SSD address space. A bounded *hub* / *shared hot set* of records is explicitly cached into the window or into CXL-DRAM (§4.3). Frontier-top neighbors are candidates for precise promotion, not for bulk 2-hop prefetch (F3).

This is a deliberate difference from DiskANN, where adjacency and vectors share SSD sectors. We pay a modest host-DRAM tax for the graph in order to stop the window from turning into an accidental adjacency cache.

4.2 D2: Bounded precise promotion

Promotion is an explicit `promote(ids[], level, budget)` on the residency plane (Figure 6). The only ids that enter the call are the neighbors of the *current frontier top-M* — the nodes the walk will expand next. A global

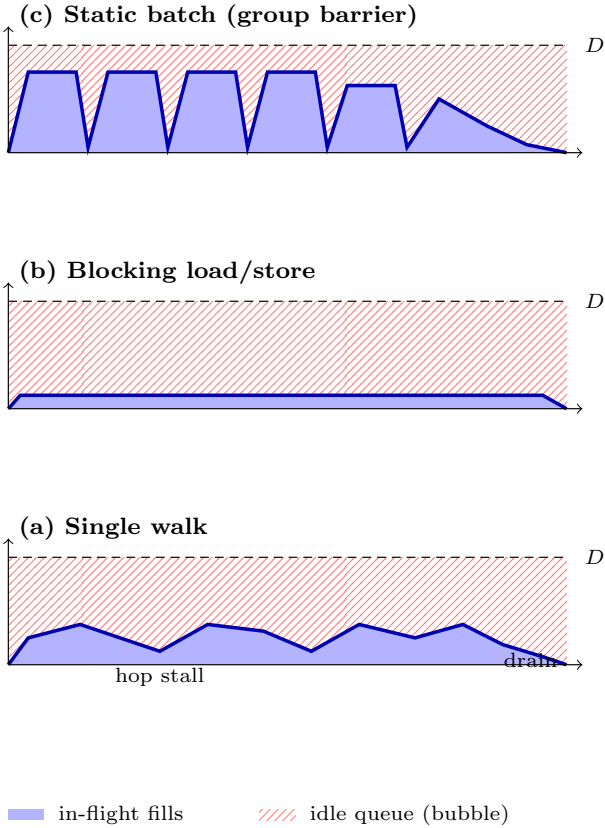


Figure 4. Pipeline bubbles on a faulting CXL address space (schematic). y is in-flight fills; D is the depth NAND parallelism needs. A single walk cannot hold D ; a blocking load/store is stuck at $QD \approx 1$; a static batch drains at the barrier. How the runtime issues across queries is §4.

and a per-query byte budget cap how much may move. `install_top` decides *who stays* in the window after a fill, not *who the hop may wait for to score*. Precision is scored-from-window/fetched, not “every fetched page later scored from a host bounce.”

The measured P3 loop still scores from that bounce after `PREFETCH_BATCH` returns (§5). That is overlap, not hide; the rewrite in the evaluation plan splits the two.

We forbid the policies F3 already falsified: synchronous high-coverage 2-hop prefetch, `install_all`, and “fault harder” loops that raise queue depth by touching unused pages. Over-promotion is the failure mode the byte budget exists to prevent.

On corpora where several vectors share a 4 KB page (low dimension), a *page-bin / reorder* layout clusters IDs that the walk co-touches so that one fill admits several useful records. This is layout serving residency, not a new index family. We report it as a placement option

and show when it helps (small dim) and when it does not (vector $\approx$ page).

4.3 D3: Entry pin and soft-pin (hot set demoted)

Single-query deepening is a bad way to spend a small window (F4, F7). A 64 MiB shared LFU set raised hit rate and *destroyed* QPS on a warm protocol ($14.3 \rightarrow 2.4$); we do not ship it as D3. What remains is a tiny hard pin of the entry subgraph (≤ 32 MiB) and a hop-decaying soft-pin of recently installed pages. Pinning is never implicit in a page fault. Cross-query reuse, if it happens, is because prefetch and `install_top` left useful pages in the window—not because an LFU daemon is competing with search.

4.4 D4: Hide engine — lookahead prefetch and continuous batching

D4 is the hide engine, not a QPS knob. Offline, we pick the smallest beam L that meets the recall@ k floor. Online, the runtime issues `PREFETCH_BATCH` for pages the *next* hop (and the next admitted query) will score, then computes distances only on bytes already in the window. Continuous fetch-level batching keeps a target device depth D across queries so one pointer-chasing walk does not collapse to $QD \approx 1$ (Figure 7). Scoring of already-resident records is never queued behind a fill it does not depend on. We do not merge the PipeANN binary; its `cont` path is the block-device contrast, not this runtime.

4.5 D5: Observability

Every experiment in §6 is required to report hide counters first: fraction of full-precision scores whose bytes were already in the window (`score_from_window`), critical-path wait on NAND (`crit_wait_ns`), overlap (`device_fill_ns/wall_ns`), then QPS at the iso-recall floor. Promote precision is scored-from-window/fetched. QPS without those counters is not a hide result.

What we refuse to hide. If a module cannot move `crit_wait_ns` or `score_from_window` in an ablation, it is removed from the contribution list rather than kept as decoration. An LFU hot set that raises hit rate and drops QPS is already in that bin.

5 Implementation

FlashANNS is a userspace beam search (`serving/search_beam`) over a CXAN packed (or page-binned) image `mmap`’d from `/dev/vmem0`. Policy P3 issues hop-ranked miss pages with `VMEM_IOC_PREFETCH_BATCH`, copies into a host bounce (`PageCopyPool`), scores there, and installs only `install_top` pages into a 1 GiB `DramWindow`. That

Fig. 6 placeholder — architecture:
 top: Placement | Window admit | Prefetch+cont | L ctrl;
 middle: residency plane (fill off crit path / score from window);
 bottom: host DRAM (graph/PQ) | window = virtual CXL-DRAM | CXL-SSD.

Figure 5. FlashANNS architecture. From the host, scored bytes look like CXL-DRAM; capacity is CXL-SSD. The residency plane is the only path that moves full-precision bytes. Search never page-faults a neighbor “to be helpful.” **Fill-in:** use shipped names (PREFETCH_BATCH, DramWindow, install_top); mark vmem_sw.

Fig. 7 placeholder — promote path:
 frontier top- M → budget check → async
 fill → window insert → precision feedback.

Figure 6. Bounded precise promotion. Only soon-to-expand neighbors are eligible; a byte budget and a live precision estimate keep the window from becoming a junkyard. **Fill-in:** annotate the implemented ioctl / batch path and the default M / budget used in §6.

split is exactly the hide debt §6: scoring still waits on the bounce, so NAND is not yet off the critical path.

What we measure (2026-08-30, gpu01). Kernel 6.18.0-rc5. FPGA Altera 0000:15:00.0 bound to ntcx/mem2nvmex (32 GiB BAR). NAND I/O goes through nvme on 0000:d9:00.0 (→ /dev/nvme4n1, Dell CD8P 1.92 TB). Userspace /dev/vmem0 is vmem_sw: 28 GiB logical RAM map, 4 GiB device cache, 2 MiB stripe. HPS registers return -EOPNOTSUPP; stores to the BAR read back as 0xff. Hardware vmem.ko implements PREFETCH_BATCH but is **not** the measured path. CXL-DRAM DAX is absent; DramWindow is anonymous pages bound to NUMA node 1.

What is implemented. P0 demand promote and P3 bounded install; PREFETCH_BATCH; page-bin layout pack; iso-recall L picker; optional graph copy into host DRAM (intended default for hide rows). Entry subgraph ≤ 32 MiB is pinned. A shared LFU hot set was prototyped and lost QPS on the warm protocol; we do not ship it as a result.

What is not implemented, and is not claimed. Product CXL.mem SSD; HPS fill of the FPGA BAR;

Shared FAM; a 160 GB/s DRAM-SSD engine; closed-loop F9 miss-chop; multi-thread shared-window speedup. DiskANN PQ-off is never placed on the PQ-on line.

Prototype scale. The serving binary is a single translation unit plus small headers on top of the CXAN layout. The software CXL-SSD path is the out-of-tree vmem_sw driver. A log header that omits kernel, BDF, and vmem_sw vs. vmem.ko is an invalid row.

6 Evaluation

We answer five questions:

- Q1** Does naive CXL-SSD graph ANNS fail in the ways §3 claimed?
- Q2** At iso-recall, does FlashANNS move FP scoring off the NAND critical path (hide), and what QPS remains vs. demand and vs. an in-window oracle?
- Q3** Which module (graph-in-host, bounded install, prefetch/cont, page-bin) moves *hide* (critical-path wait, score-from-window), not only QPS?
- Q4** Does hide appear only after warmup (service hide), or also on a true-cold reload?
- Q5** What remains vs. an in-window oracle and vs. PipeANN, and is the residual honest?

6.1 Setup

Table 4 is the locked configuration. The primary metric is *iso-recall QPS* (and p99) at a stated recall@ k floor. We always report NAND reads, promote bytes, and the three-level hit breakdown. A run that does not declare cold vs. warm is discarded: the window is small enough that accidental warm-up invents speedups.

Baselines. (1) **Demand:** CXL-SSD load/store, no promote. (2) **Page:** 16 KB admission. (3) **Sync-2hop:** the F3 policy. (4) **Static-wide:** large beam, no controller. (5) **Miss-chop:** cut beam on instantaneous miss (the F9 policy). (6) **DiskANN/PipeANN:** official block path, same machine, same corpus, same recall floor; PQ

Fig. 8 placeholder — miss pipeline:
 queries → fetch-level scheduler (target D) → async CXL-SSD fills → host score → admit next.
 Show why blocking load/store cannot hold D .

Figure 7. Continuous fetch-level batching over asynchronous memory-semantic fills. Recall is unchanged by construction: prefetch only changes *when* a needed page is resident, not *which* nodes the walk visits. **Fill-in:** draw the implemented **cont** path; put the blocking-mmap failure mode on the same figure.

Table 4. Locked evaluation platform (2026-08-30, gpu01).

Item	Configuration
Host CPU / DRAM	dual-socket Xeon, ~64+64 GiB host DRAM
CXL-DRAM window	absent ; 1 GiB DramWindow = mbind node 1
CXL-SSD	FPGA nvme+CD8P; /dev/vmem0=vmem_sw
Window / cache	1 GiB host window, 4 GiB vmem_sw cache
Block-SSD baseline	PipeANN on /mnt/disk0 (nvme0n1), PQ on
Primary corpus	Text2Image-10M, 200-d, MIPS
Sensitivity	packed vs. page-bin; L picker
Index	Vamana $R=32$, oneshot FP (no PQ nav on service)
Target quality	recall@10 ≥ 0.92 (picker chose $L=400$)
Protocol	cold: rmmmod vmem_sw until cache_used = 0

on unless a figure is labeled `--no_pq_nav`. (7) **Oracle-in-window:** pin the measured working set in the 1 GiB window, then search with no SSD fills (the hide upper bound). This is not DiskANN.

We never present a home-grown `O_DIRECT` reader as “DiskANN.”

6.2 Q1: Naive CXL-SSD fails

Figure 2–4 are reproduced on the evaluation machine under the Table 4 protocol so that Design and Evaluation share one dataset. We expect the same qualitative shape: a two-order cliff, page admission and blind prefetch losing to demand, beam cooling the set, and blocking load/store stuck near $QD \approx 1$. If a finding does not reproduce, it is dropped from the intro, not explained away in a footnote.

6.3 Q2: End-to-end iso-recall (the money plot)

Figure 8 and Table 5 are the submission’s main result. On the 2026-08-30 true-cold T2I-10M boot (clock alloc, miss-only, expand-bundle pages), hide pagebin is 50.25 QPS at $nq=20$ and 49.25 at $nq=100$ ($L=400$, recall@10 ≥ 0.92 , **from_win**=100%, bounce = 0, in-page slot-use 100% with no extra scores). Neighbors of one expand share seven sequential pages; we score only those neighbors. Star-BFS pagebin without the bundle is 25%

slot-use (one of five). Real NVMe occupancy is sectors-read $\times$ 512 over the timed window over the isolated random **PREFETCH_BATCH** peak (1.560 GB/s), not “a fill is in flight.” One query is $0.600/1.560 = 38.5\%$; four concurrent queries (private 512 MiB windows, $nq=100$) reach $1.027/1.560 = 65.8\%$ at 87.21 QPS—the useful-only ceiling ($\approx 12 \text{ MB/q} \times 87 \text{ QPS} \approx 1.04 \text{ GB/s}$). That is score-path hide; single-query service hide vs. the in-window oracle (85.8 QPS) is $50.25/85.8 \approx 0.59\times$. QPS vs. PipeANN (57.6 at $L=2000$, PQ on, $T=1$) stays a named residual—oneshot FP vs. PQ navigation, not a hidden base. We write the residual as a gap to the oracle, not “search cannot see flash.”

6.4 Q3: Each module is necessary

Figure 9 and Table 6 are the falsifiability test for contribution 3. We also sweep promote M and hot-set fraction to show the F3 lesson in a positive form: there is a budget at which precision and QPS peak, and past it they fall.

6.5 Q4: Cold hide versus service hide

Figure 10 is the service question. A production ANNS process is allowed a discarded warmup; a paper row is not allowed to confuse that with a true-cold reload. If **crit_wait** collapses only on the warm pass, we say so in the caption and do not migrate the sentence into §1. The one-copy capacity sketch of §2 remains a model; we do not plot it as a measured Q4.

6.6 Q5: Losses, overhead, and fairness

We report the cases FlashANNS loses or looks worse: warm working sets that already fit the window (async prefetch $\approx$ demand); low dimension where page $\approx$ record; the residual gap to PQ-navigated DiskANN; and software-proxy artifacts (global cache lock, QD ceiling). Host-DRAM consumed by the graph and PQ is a cost, not a footnote. CPU is reported because DiskANN on this class of machine is I/O-bound, not CPU-bound; if FlashANNS becomes CPU-bound we say so.

Fig. 9 money plot (hide first):
 stacked `crit_wait` vs. compute at iso-recall;
 twin axis QPS. Lines: Demand, P3 packed, P3 pagebin,
 –prefetch, oracle-in-window, PipeANN $T=1$.

Figure 8. Hide at the primary recall floor. The money plot is critical-path NAND wait, not QPS alone. FlashANNS passes only if `crit_wait` approaches the in-window oracle. QPS vs. PipeANN is a named residual (oneshot FP vs. PQ navigation). Until hide counters pass we write “gap to demand,” not “search cannot see flash.”

Table 5. True-cold T2I-10M, $L=400$, $nq=100$, seed 42, oneshot FP. Hide cells (`crit_wait`, score-from-window) stay — until the runtime split. NAND/promote columns stay — until remeasured.

System	Recall@10	QPS	p99 (μ s)	NAND reads/q	Promote B/q	Host DRAM
Demand CXL-SSD (P0)	0.944	1.64	1.3×10^6	—	—	1 GiB win.
Page admission	—	—	—	—	—	—
Sync 2-hop	—	—	—	—	—	—
Static wide beam	—	—	—	—	—	—
Miss-chop beam	—	—	—	—	—	—
DiskANN (PQ on)	—	—	—	—	—	—
PipeANN pipe $T=1$	0.925	57.6	1.7×10^4	—	—	PQ nav
Oracle host DRAM ($nq=20$)	0.950	66.5	4.8×10^4	0	—	9.6 GiB pop.
Oracle 1 GiB window ($nq=20$)	0.925	85.8	1.5×10^4	0	—	WS pinned
FlashANNS hide packed ($nq=20$)	0.930	24.67	5.7×10^4	—	100% win	1 GiB
FlashANNS hide pagebin ($nq=20$, $L=400$)	0.940	50.25	2.7×10^4	—	100% win	1 GiB
FlashANNS hide pagebin ($nq=100$, $L=400$)	0.933	49.25	3.6×10^4	—	100% win	1 GiB
FlashANNS P3 packed ($nq=100$, pre-hide)	0.944	4.08	2.4×10^5	—	—	1 GiB
FlashANNS P3 pagebin ($nq=100$, pre-hide)	0.932	5.35	1.9×10^5	—	—	1 GiB

Every — cell is a placeholder. Replace with the measured value under the protocol in §6.1.

Fig. 10 placeholder — two panels:
 (a) leave-one-out *hide* (`crit_wait`, score-from-window) and QPS (full vs. –promote
 –prefetch –pagebin –graph-in-DRAM); do not plot –LFU-hotset as a loss;
 (b) promote-byte / hot-set-fraction Pareto (QPS vs. budget).

Figure 9. Ablation and budget Pareto. Removing any contributed module must move the main metric. The Pareto must show a bounded sweet spot, not “more cache is always better.” **Fill-in:** same recall floor as Figure 8.

Takeaway. Q2–Q4 must be answerable from hide counters, not from QPS prose. If the money plot needs a verbal caveat that changes the claim, the claim in §1 is the thing to change.

7 Related Work

Table 7 is the comparison we want a reviewer to use. Prose below only records the one-line difference; it does not repeat the background.

Block-SSD and distributed graph ANNS.. DiskANN, FreshDiskANN, PipeANN, and SPANN [9, 12, 18, 19]

Table 6. Leave-one-out at the primary recall floor.

Variant	QPS	crit_wait
Full FlashANNS (pagebin)	5.35	—
— page-bin (packed)	4.08	—
— <code>PREFETCH_BATCH</code>	—	—
— graph-in-host	—	—
— bounded <code>install_top</code>	—	—
Demand only (P0)	1.64	NAND-class
LFU hot set (void)	—	—

Every — cell is a placeholder. Replace with the measured value under the protocol in §6.1.

Fig. 11 — cold reload vs. 1 discarded warmup:
 y : `crit_wait`, `score-from-window`, QPS.
 If only warm hides, label it service hide.

Figure 10. True-cold (`vmem_sw cache_used` = 0) versus a discarded warmup then 100 queries. Hide that appears only after warmup is a service property, not a cold-start property. Shared-FAM provisioning stays a discussion model, not this figure.

optimize sector layout, PQ navigation, and I/O overlap on a block device. DistributedANN [6] serves one logical graph from a KV store. FlashANNS is not “a better `io_uring` scheduler.” Its first-class objects are a CXL-DRAM-shaped window and the fills that must finish before a hop scores. NVMe-oF and KV sharing still present a block/KV slow path and a per-host cache.

CXL-DRAM and near-memory ANNS. CXL-ANNS and Cosmos [4, 14, 15] — and second-tier / residual designs [1, 11, 17] — assume a DRAM- or NVM-priced far tier and optimize for ~ 100 ns, not for tens of microseconds of NAND plus a small polluted window. Their caches and prefetchers are not transferable at that delay gap (F3). We do not require Type-2 / PNM compute.

CXL-SSD hardware and generic software. Sky-Byte, From-Block-to-Byte, CMM-H, and CXL-AnySSD [3, 5, 7, 10] build or evaluate the medium. They do not jointly manage graph-ANNS residency, bounded promotion, and stall-aware search. P-HNSW [8] is about crash consistency on PM, not serving residency on CXL-SSD.

8 Discussion and Limitations

Proxy versus product. The measured path is FPGA `From-mem` plus `vmem_sw`, not a shipping CXL.mem SSD. HPS does not fill the 32 GiB BAR from the host CPU; we will not relabel that BAR as the evaluated window. Latency must fallide and QD collapse are still properties of memory-semantic flash.

Hide is the service contract, not a fallback.

A usable ANNS service on this tier means the scoring thread does not wait on NAND: every full-precision distance is taken from bytes already in the window (`score_from_window` = 100%), and `crit_wait_ns` is DRAM-class. Graph/PQ live in host DRAM; entry 2-hop is warmed off the timed path; lookahead plus continuous fills keep the next hop resident. A window miss while scoring is a hide failure we close, not a reason to shrink the claim.

One-copy is a deployment claim, not a result.

Figure 10b is a provisioning model. Shared FAM is unmeasured. Device IOPS remain shared even with one copy.

DiskANN/PipeANN remain the block-path champions. PQ navigation plus deep aio avoids most full-precision reads. FlashANNS oneshot-FP on a load/store flash mapping is a different contract. A $\sim 11\times$ QPS gap to PipeANN $T=1$ at iso-recall is expected and reported.

Hot set and `install.all`. A 64 MiB shared LFU set raised hit rate and destroyed QPS (window thrash). D3 in the paper is entry pin + soft-pin. True-cold `install.all` did not reproduce a historical $\div 5$ QPS drop; we report traffic, not a fake collapse.

What would change our mind. If HPS makes the FPGA BAR a coherent miss window, we re-measure hide there and keep the same counters. If hide counters never leave NAND-class wait, contribution 3 shrinks to “overlap vs. demand” and the intro must not say the host is unaware of CXL-SSD.

9 Conclusion

Graph ANNS has a well-engineered block-SSD path and a well-engineered CXL-DRAM path. Flash-backed CXL memory is the third deployment point — flash-priced capacity, a memory-semantic window, and a future one-copy hosting story — but it is unusable if treated as slow DRAM or as mmap’d DiskANN. FlashANNS is a runtime for that tier: the host should score from a small window as if it were CXL-DRAM, while prefetch and continuous batching hide NAND that actually holds D . The opportunity is the medium; the contribution is making flash invisible on the critical path. The contribution is making flash invisible on the critical path: 100% of

Table 7. Related systems on the dimensions that actually differ. **Fill-in:** keep these columns; add a row only if it changes a column.

System	Full-prec. home	Graph home	Slow path	Capacity assumption	Optimize for	Cache unit
DiskANN / PipeANN	NVMe sectors	with vectors	async block I/O	flash is enough	IOPS, PQ trips	4 KB / entry cache
Second-Tier / FaTRQ	CXL/RDMA/NVMe	varies	far load	second tier holds index	4 KB vs. fine read	fine-grained far
CXL-ANNS	all CXL-DRAM	CXL	CXL.mem + DSA	pool holds corpus	~100 ns + near-data	relation cache
Cosmos / PNM	device DRAM	device	on-device cores	multi-device DRAM	offload walk/dist	device place
CXL-SSD HW / AnySSD	NAND HDM	n/a	fault / generic	flash + window	composable SSD, no swap	pages / OS
FlashANNS	CXL-SSD	host DRAM	fault + async fill	flash required; not all-DRAM	residency + stall	record / hub / top-M

scores come from the window, and the host does not wait on CXL-SSD. The remaining work is to keep that invariant under load, not to retreat from it.

References

- [1] Anonymous. 2024. Second-Tier Memory for Vector Search. *arXiv preprint* (2024). Replace with the Second-Tier Memory citation..
- [2] Anonymous. 2025. Bauhaus: CXL Memory Tiering and Stranded Capacity. *Placeholder* (2025).
- [3] Anonymous. 2025. CMM-H: CXL Memory Module with NAND Backing. *Placeholder* (2025). Replace with the CMM-H citation..
- [4] Anonymous. 2025. Cosmos: CXL-Based Full In-Memory Approximate Nearest Neighbor Search with Near-Data Processing. *IEEE Computer Architecture Letters* (2025). Replace with the camera-ready Cosmos / CAL’25 citation..
- [5] Anonymous. 2025. CXL-AnySSD: Composable CXL SSDs without Swapping. (2025). UIUC thesis; replace with the public citation..
- [6] Anonymous. 2025. DistributedANN: Serving a Logical DiskANN Graph over a Distributed Key-Value Store. *arXiv preprint arXiv:2509.06046* (2025).
- [7] Anonymous. 2025. From Block to Byte: Rethinking SSDs with CXL. In *Placeholder*. Replace with the From-Block-to-Byte citation..
- [8] Anonymous. 2025. P-HNSW: Crash-Consistent Hierarchical Navigable Small World Graphs on Persistent Memory. *Placeholder* (2025).
- [9] Anonymous. 2025. PipeANN: Pipelined Graph-Based Approximate Nearest Neighbor Search on SSD. In *USENIX Symposium on Operating Systems Design and Implementation*. Replace with the camera-ready PipeANN / OSDI’25 citation..
- [10] Anonymous. 2025. SkyByte: Architecting an Efficient Memory-Semantic CXL-based SSD. In *Placeholder*. Replace with the SkyByte citation..
- [11] Anonymous. 2026. FaTRQ: Far-Memory-Aware Residual Quantization for Vector Search. *arXiv preprint* (2026). Replace with the camera-ready FaTRQ citation..
- [12] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021.

- SPANN: Highly-efficient Billion-scale Approximate Nearest Neighbor Search. In *Advances in Neural Information Processing Systems*.
- [13] CXL Consortium 2023. *Compute Express Link Specification, Revision 3.1*. CXL Consortium.
 - [14] Junhyeok Jang, Hanjin Choi, Hanyeoreum Bae, Seungjun Lee, Miryeong Kwon, and Myoungsoo Jung. 2023. CXL-ANNS: Software-Hardware Collaborative Memory Disaggregation and Computation for Billion-Scale Approximate Nearest Neighbor Search. In *USENIX Annual Technical Conference*.
 - [15] Junhyeok Jang, Hanjin Choi, Hanyeoreum Bae, Seungjun Lee, Miryeong Kwon, and Myoungsoo Jung. 2024. Bridging Software-Hardware for Billion-Scale ANNS with CXL Memory Disaggregation. *ACM Transactions on Computer Systems* (2024).
 - [16] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. In *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
 - [17] Jie Ren, Minjia Zhang, and Dong Li. 2020. HM-ANN: Efficient Billion-Point Nearest Neighbor Search on Heterogeneous Memory. In *Advances in Neural Information Processing Systems*.
 - [18] Aditi Singh, Suhas Jayaram Subramanya, Rohan Kadekodi, and Harsha Vardhan Simhadri. 2021. FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search. In *arXiv preprint arXiv:2105.09613*.
 - [19] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems*.

A Fill-in Checklist

Table 8 is the author-facing map from each placeholder to the result file that should replace it. Delete this appendix before a submission if it is still author notes; or keep a cleaned artifact appendix after the numbers exist.

Table 8. Placeholder → intended source. Author notes; not a scientific result.

Id	Type	Replace with
Fig. 1	figure*	TikZ 3-column deployment (done)
Fig. 2	figure	Log-y bars: cold / soft / warm (LAION-200k F8)
Fig. 3	figure*	Page vs. record; blind k -hop; beam as unwise
Fig. 4	figure	TikZ: single walk / $QD \approx 1$ / static drain
Tab. 2	table	3-row C1–C3 (done; no —)
Fig. 5	figure*	Implemented component names
Tab. 3	table	Byte counts on the primary corpus
Fig. 6	figure	Promote API + default M /budget
Fig. 7	figure*	cont + async fill; blocking mmap failure
Tab. 4	table	Locked platform and corpora
Fig. 8	figure*	Iso-recall QPS (money plot)
Tab. 5	table*	End-to-end row per baseline
Fig. 9	figure*	Leave-one-out + Pareto
Tab. 6	table	Leave-one-out numbers
Fig. 10	figure	Scale sweep + provisioning model
Tab. 7	table*	Camera-ready citations

B Artifact Outline

An ASPLOS artifact should include: the identity gate for the CXL-SSD node; scripts that reset the window (cold

protocol); the iso-recall harness; and the exact command lines that produce Figures 8–10. We will not point a reviewer at an identifiable internal hostname.